Building and Analyzing Monolingual Transformer LMs in Two Indian Languages

Roll No – 2025201029

Model H = Hindi (higher-resource).
Model L = Nepali (lower-resource, picked from the assignment's allowed list).
We built *two* decoder-only Transformer language models from scratch in PyTorch. They never share data, a tokenizer, or weights, at any point across the three phases.

1 Phase 1: Data Collection and Tokenizer Construction

Before training, each language needed its own large, clean text and its own way to split that text into small pieces. We built two corpora, one per language, each combining downloaded text with news we scraped ourselves.

Where the data came from. For **Hindi**, we downloaded IndicCorp v2 and scraped 7 Hindi news sites, including `jagran.com`, `abplive.com`, and `livehindustan.com`. For **Nepali**, we downloaded the same IndicCorp v2 and scraped 15 Nepali news sites, including `onlinekhabar.com`, `ekantipur.com`, and `setopati.com`. News scraping alone left Nepali short of Hindi's size, so we added one more source: a full crawl of Nepali Wikipedia (`ne.wikipedia.org`).

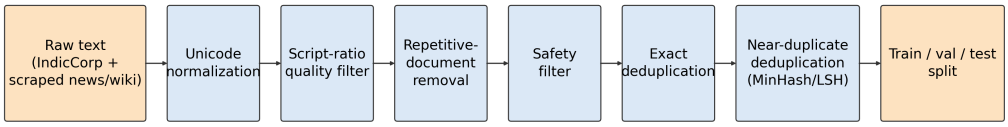


Figure 1: Phase 1 preprocessing pipeline.
Each box fixes one specific problem in the raw text, and every step runs separately for each language. The two shaded boxes are the two ends of the pipeline: raw text goes in on the left, a clean train/val/test split comes out on the right.

Step	What it does
Unicode normalization	Makes sure the same letter is always written the same way, so the computer never confuses two characters that look identical.
Script-ratio filter	Keeps a page only if most of its text is really in Devanagari, and throws away pages that are mostly English, numbers, or website junk.
Repetitive-doc. removal	Throws away pages that just repeat the same phrase again and again - usually spam or website boilerplate, not real content.
Safety filter	Checks each page against a list of bad words and removes any page that contains them.
Exact deduplication	If two pages are exactly the same, keeps only one copy.
Near-dup. deduplication	Uses a technique called MinHash to find pages that are almost the same, like one news story posted on two different sites, and removes the extra copy.

Table 1: Preprocessing techniques, in the order we apply them.
Every row runs independently per language - nothing from Hindi's corpus ever touches Nepali's, or the other way round.

	Hindi (H)	Nepali (L)
Final corpus size (tokenizer estimate)	506.81M tokens	500.24M tokens
Manual fraction	21.07%	20.04%
Manual sources	News scrape	News scrape + Wikipedia
Train / val / test split	95% / 2.5% / 2.5%	95% / 2.5% / 2.5%
Tokenizer	SentencePiece BPE, vocab 10,000	SentencePiece BPE, vocab 10,000
Fertility (tokens/word)	1.3227	1.4934
Avg. characters/token	3.856	4.318
UNK rate (byte-fallback enabled)	0.0%	0.0%

Table 2: Phase 1 corpus and tokenizer statistics.
Both corpora land close to the assignment's ∼500M-token target, and both tokenizers hit a 0% unknown-token rate.

Even with Wikipedia added, Nepali was still harder to collect cleanly. Our near-duplicate check found more repeated content in Nepali's scraped pages than in Hindi's, about 6.2% on the Wikipedia portion alone. In short, there is less distinct, freely available Nepali text on the internet, and reaching Hindi's token count took real extra effort, not just extra scraping time.

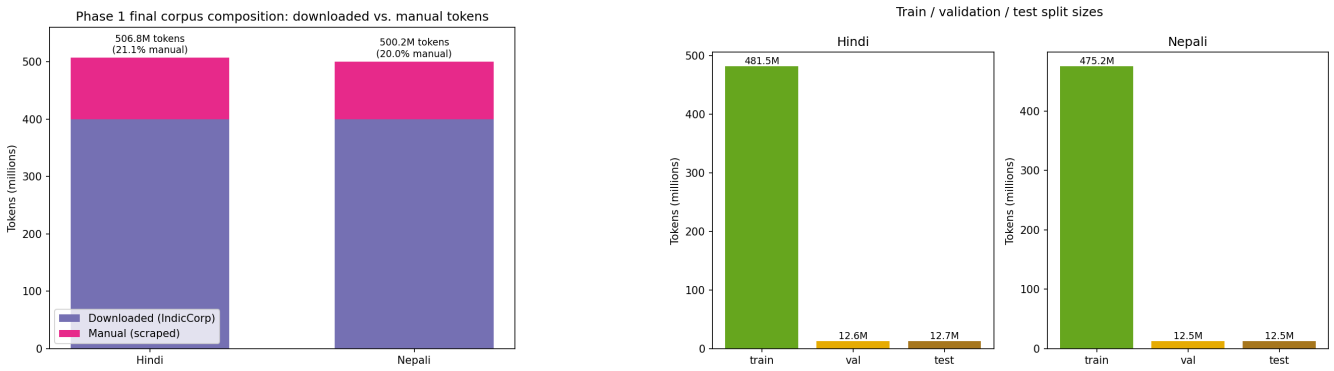
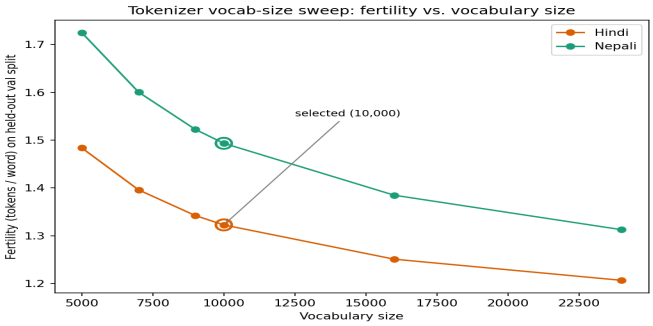


Figure 2: Corpus composition and split sizes.

We trained both tokenizers from scratch, never shared between the two languages. We tried several vocabulary sizes (5k–24k, Figure 3) and picked 10,000, balancing better fertility against our Phase 2 parameter budget instead of just choosing the largest size. With `byte_fallback=True` turned on, every size gave a 0% unknown-token rate: any character the tokenizer has never seen just falls back to raw bytes instead of an `<unk>` token.



Vocab	Hindi	Nepali
5,000	1.4839	1.7252
7,000	1.3960	1.6005
9,000	1.3421	1.5224
10,000	**1.3227**	**1.4934**
16,000	1.2509	1.3848
24,000	1.2067	1.3126

Figure 3: Tokenizer fertility vs. vocabulary size.
Left: fertility (tokens/word) at every vocabulary size we tried, shown as a curve. Right: the same numbers as a table. Fertility keeps dropping as the vocabulary grows, but the gains shrink quickly - the 10,000 we chose (bold) sits past the steepest part of the curve, a deliberate trade-off against the parameter budget.

2 Phase 2: Model Architecture, Pretraining, and Evaluation

With two clean, tokenized corpora ready, this phase is where the language models actually get built and taught to write.

2.1 Architecture

We built a decoder-only (GPT-style) Transformer using only basic PyTorch pieces: `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout`. We did not use `nn.Transformer`, HuggingFace, or any ready-made attention block - we wanted to understand and control every part of the model ourselves. Both languages use the same architecture; only the trained weights are different.

Hyperparameter	Value
`vocab_size`	10,000 (each language's own tokenizer)
`d_model` / `n_layers` / `n_heads`	512 / 6 / 8 ($d_{head} = 64$)
`d_ff`	2,208 (chosen to land on the ∼25M-parameter target exactly)
`context_length`	256
Positional encoding	RoPE (Rotary Position Embedding)
Activation / Norm	GELU / Pre-norm
Weight tying	Yes (input embedding ↔ output head)
Dropout (embed / attn / residual)	0.1 / 0.1 / 0.1
Parameter count	**25,019,328** (both models)

Table 3: Phase 2 model configuration.
Both languages use this exact same shape; only the learned weights differ.

Positional encoding. On its own, self-attention does not know word order - it would treat a sentence like a random bag of words unless something tells it the position of each word. RoPE fixes this by rotating the query (Q) and key (K) vectors based on position, and it does this in every attention layer, not just once at the start. Its position tables are fixed in advance up to `context_length`, so a longer sentence raises an error instead of being guessed - this is why our model's maximum sentence length is 256 tokens. We chose RoPE because it is the standard choice in most major open LLMs since 2022 (Llama, Mistral, Qwen), and because it helps with the comparison tasks in Phase 3.

Multi-head causal attention. We turn each word into three vectors - Q, K, V - using one combined layer, then split them into multiple heads, each looking at the sentence in its own way. RoPE is applied to Q and K only. Each head then compares its Q and K vectors to decide what to focus on, and divides the result by $\sqrt{d_{head}}$ to keep the numbers stable - without this, the scores would grow too large and the model would get stuck focusing on just one word every time. We also add a mask so that word t can only look at word t or earlier, never a word that comes after it. We checked this really works: changing a later word never changed the output for an earlier position.

Transformer block. Each block runs the same six steps, in order:

1. Normalize the input (LayerNorm, applied *before* the sub-layer - this is what "pre-norm" means).
2. Run multi-head causal self-attention on the normalized input.
3. Add the attention output back onto the original, un-normalized input (a residual connection).
4. Normalize again, with a second, independent LayerNorm.
5. Pass the result through a feed-forward layer: a linear layer up to a wider hidden size, a GELU activation, then a linear layer back down to d_{model}.
6. Add the feed-forward output back onto the residual stream from step 3.

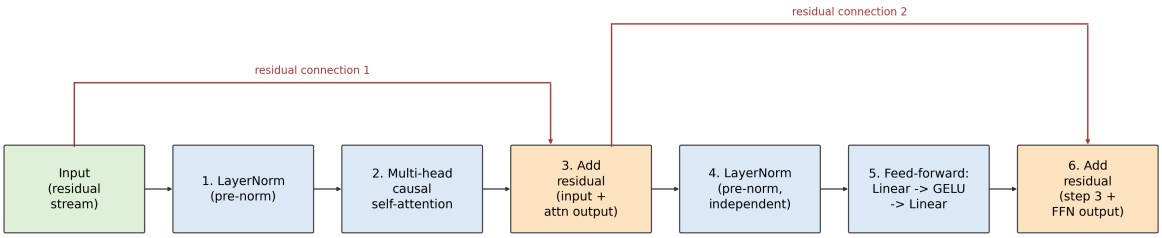


Figure 4: The six steps of one Transformer block, left to right.

The two dark-red lines are the residual (skip) connections: they carry the un-normalized signal straight past attention and past the feed-forward layer, so it is only ever *added to*, never replaced.

We used pre-norm instead of post-norm because it keeps the residual path clean, letting gradients flow back through many layers without shrinking or exploding. The output layer also reuses the input embedding's weights instead of learning a separate one - a common trick called weight tying - saving about a fifth of our entire 25M parameter budget. In simple terms: the same table that turns a word into numbers at the start is reused to turn numbers back into word guesses at the end, instead of building two separate tables for the same job.

2.2 Pretraining

We trained with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$), warming up the learning rate for the first 1,000 steps, then slowly lowering it from 3e−4 to 3e−5 over 30,000 steps total - roughly one full pass through each corpus. We saved a checkpoint every 1,000 steps throughout training.

AdamW updates each weight θ using:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{m_t}{\sqrt{v_t} + \epsilon} + \lambda \theta_{t-1} \right)$$

where g_t is the gradient, m_t / v_t are running averages of the gradient and its square, η is the learning rate, and $\lambda \theta_{t-1}$ is the weight-decay term that keeps weights from growing too large.

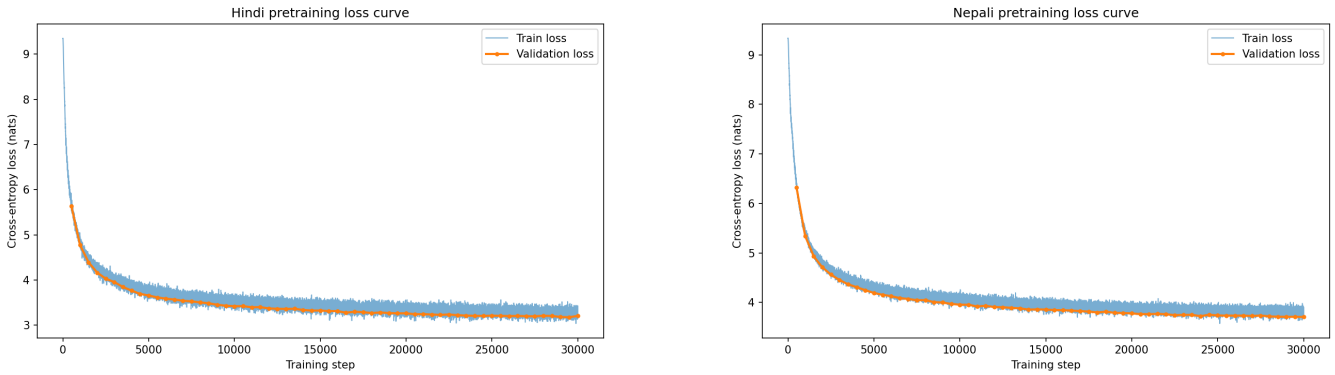


Figure 5: Pretraining loss curves: Hindi (left) vs. Nepali (right).

Both curves start near $\ln(10{,}000) \approx 9.2$ nats - exactly the loss of a completely untrained model guessing randomly over 10,000 tokens - and flatten out by step 30,000. Train and validation loss track each other closely the whole way for both languages, which tells us neither model is overfitting; that is expected for a 25M-parameter model on a ∼500M-token corpus.

2.3 Language-Modeling and Generation Evaluation

Nepali's perplexity score looks much worse than Hindi's, but that is mostly a tokenizer effect, not a real quality gap.

	Hindi (H)	Nepali (L)
Validation cross-entropy (nats)	3.1880	3.7115
Validation perplexity	24.24	40.91
Validation bits-per-byte (BPB)	0.4665	0.4688
BLEU-4 / chrF++ / ROUGE-L (greedy)	38.34 / 48.94 / 0.242	37.42 / 48.77 / 0.067
Distinct-1 / Distinct-2	0.300 / 0.548	0.381 / 0.556
Repetition rate (4-gram)	0.013	0.017

Table 4: Phase 2 intrinsic and generation-quality metrics.
Every metric reported for both models, side by side, on their own held-out sets.

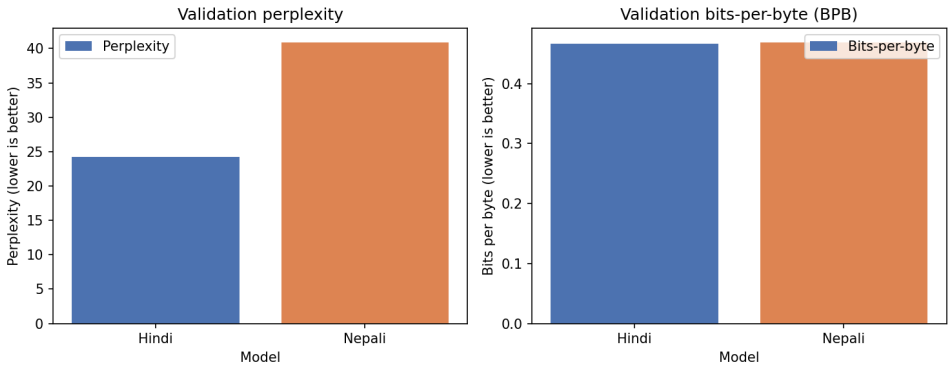


Figure 6: Perplexity vs. bits-per-byte, Hindi vs. Nepali.
Perplexity looks like a big gap between the two languages, but bits-per-byte - the metric that does not depend on the tokenizer - is under 0.5% apart.

- Perplexity is scored per token, and Nepali needs more tokens per word than Hindi (1.49 vs. 1.32, same vocabulary size), so it gets more chances to be "wrong" - pushing perplexity up even when the model is equally good.
- Bits-per-byte (BPB) scores raw text bytes instead of tokens, so it treats both tokenizers the same way. On BPB, Hindi and Nepali land within 0.5% of each other - almost identical. So the perplexity gap was mostly an illusion caused by tokenization.

Is a perplexity of 24–41 actually good? Yes - here is why it looks high but is not a problem:

- A totally untrained model guessing randomly among our 10,000 tokens has perplexity ≈10,000. Landing at 24–41 is a huge improvement, just not down to "always certain," which no real model reaches.
- Our model is small on purpose: 25M parameters, versus 124M for even the smallest GPT-2, and far more for today's large commercial models.
- We trained on about 500M tokens for roughly one pass - a fraction of what big LLMs see.

The ROUGE-L gap has a separate cause: greedy decoding.

- Greedy decoding always picks the single most likely next word. Under this, Nepali's model gets stuck repeating one memorized training sentence - real news text naming Nepal's then-Prime Minister, KP Sharma Oli - regardless of the prompt.
- With temperature sampling instead, Nepali's outputs get noticeably more varied and the repeated sentence stops dominating.

Setting	Hindi: BLEU / chrF / ROUGE-L	Nepali: BLEU / chrF / ROUGE-L
Greedy	38.34 / 48.94 / 0.242	37.42 / 48.77 / 0.067
Temp 0.5	38.99 / 49.00 / 0.219	37.75 / 48.96 / 0.067
Temp 1.0	38.59 / 47.83 / 0.250	37.85 / 49.10 / 0.067
Temp 1.5	37.67 / 46.49 / 0.226	36.14 / 47.06 / 0.043

Table 5: Generation quality at each decoding setting.
Same 30 held-out prompts, scored the same way, just with a different decoding rule each time.

- These scores barely change between greedy and temperature 0.5 or 1.0. Only temperature 1.5, the most random setting, brings them down a little.
- So temperature does not fix ROUGE-L - it fixes the repetition problem instead, which we confirmed by reading the raw generated text.

2.4 Attention Analysis

Layer	0	1	2	3	4	5
Hindi mean entropy (bits)	0.913	1.052	1.043	1.132	1.068	1.067
Hindi mean distance (pos.)	0.885	0.919	0.895	1.102	1.089	1.072
Nepali mean entropy (bits)	0.892	1.107	1.133	1.102	0.985	0.970
Nepali mean distance (pos.)	1.000	1.044	1.288	1.250	**1.762**	**1.545**

Table 6: Per-layer attention entropy and distance, Hindi vs. Nepali.
Both languages' distance rises gradually with depth, but Nepali's jumps noticeably higher in layers 4–5 (bold), while Hindi's stays fairly flat across all six layers.

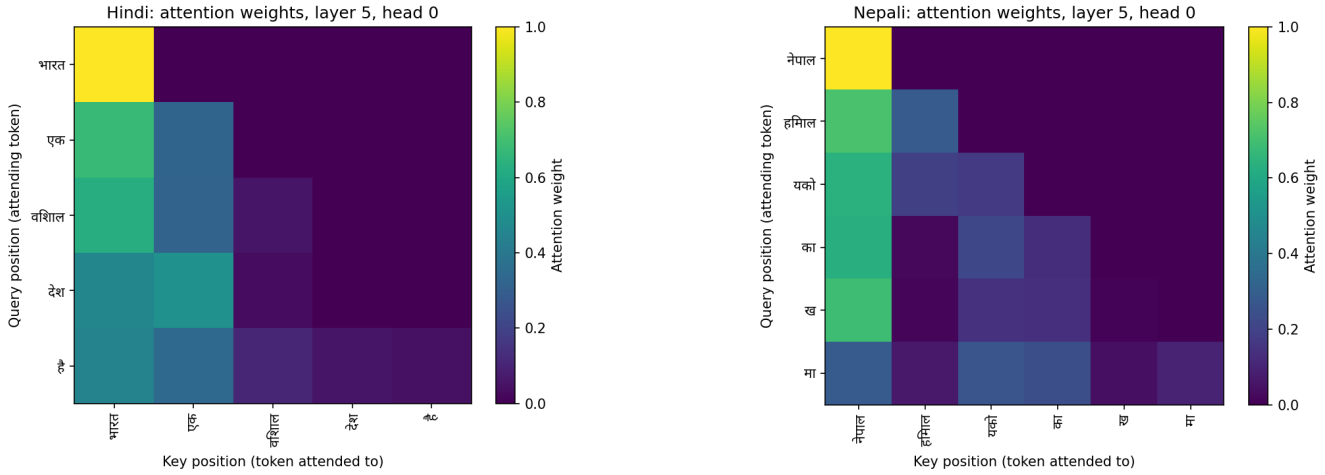


Figure 7: Layer 5, head 0, pretrained: Hindi (left) vs. Nepali (right).
Observation: in both languages, nearly every query sends most of its attention back to the very first word of the sentence - the subject - rather than to itself or its neighbours, and it looks essentially the same for both languages.

Both models show the same pattern: early layers stay tightly focused nearby, and later layers gradually look further away.

- Hindi's most focused head overall (lowest entropy) is layer 0, head 7.
- Hindi's most local head overall (lowest distance) is layer 1, head 0.
- Both of these are early layers. This fits a simple idea: early heads specialize in nearby, positional patterns, while later heads pull in wider context from across the sentence.

3 Phase 3: Reasoning Finetuning and Attention Analysis

In this phase, we test whether these models can actually reason, not just write fluently. We build a small set of comparison questions (like "who is taller, A or B?"), finetune each pretrained model to answer them, and then check whether it can answer correctly on names and numbers it has never seen before. If it can, that shows real reasoning, not just repeating what it memorized.

3.1 Synthetic Reasoning Dataset

We wrote our own question generator (one per language, seed 1337, fully reproducible), producing three kinds of comparative-reasoning questions, matching the assignment's own examples: **pairwise** (compare two entities on one attribute), **transitive3** (which of three is biggest or smallest), and **chain** (given $A>B$ and $B>C$ as separate facts, is A or C bigger). The attributes are height, age, price, and quantity. Each language gets 5,000 examples (4,000 train / 500 val / 500 test), built from 16 templates and a pool of 50 people and 20 object names. Since our script generates every number itself, we always know the correct answer for certain.

Leakage control: we split the name pool into separate train and test groups before generating any examples, so every test question uses a name the model has never seen before. We checked directly - the overlap is zero.

3.2 Finetuning

We start from each language's Phase 2 pretrained checkpoint, with its tokenizer and vocabulary kept fixed. Each example is the question followed by the answer, but when computing the loss, we skip every question token and only score the answer - so the model is never penalized for how it continues the question, only for getting the answer right. We train for 1,000 steps (8 epochs) with AdamW, using a lower learning rate than pretraining (1e-4 vs. 3e-4) and lighter weight decay (0.01 vs. 0.1), since finetuning an already-good model needs smaller, gentler updates. We save a checkpoint every 250 steps, same as before.

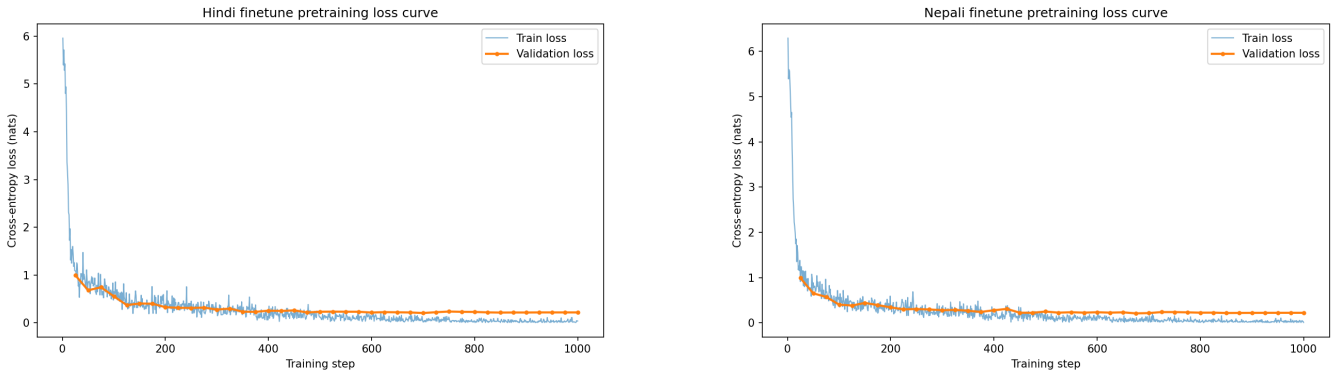


Figure 8: Finetuning loss curves: Hindi (left) vs. Nepali (right).
Training loss drops almost to zero for both languages while validation loss settles around 0.21–0.22 and stays flat. That is expected, not worrying: the answer is always one short name from a small fixed set, and a flat validation loss, rather than a rising one, tells us neither model is badly overfitting, even though both fit the training data very tightly.

3.3 Results: Pretrained vs. Finetuned Reasoning Accuracy

Before finetuning, both models score 0% - not because they cannot reason, but because they were never taught to answer with just one short name, so strict exact-match scoring gives them no credit for their usual long, wordy replies. After finetuning, both models score well above zero, even on names they have

	Hindi (H)	Nepali (L)
Pretrained-only accuracy	0.0%	0.0%
Finetuned accuracy	**65.0%**	**43.4%**
pairwise / transitive3 / chain	50.8 / 41.4 / 100.0%	42.1 / 42.1 / 45.9%

Table 7: Reasoning accuracy, pretrained vs. finetuned.
Exact match, greedy decoding, 500 test examples per language, all on held-out names.

never seen before. That is the real proof of learning to reason instead of just memorizing: a memorized name-answer pair cannot help on a name the model has never encountered.

Hindi ends up clearly ahead of Nepali, especially on chain reasoning (100% vs. 46%). The likely reason is simple: chain questions only come in 4 template variations, few enough that Hindi's finetuning could fully master them, while the more varied pairwise and transitive3 questions stay harder for both languages.

Lang.	Prompt (shortened)	Gold	Predicted
H	संजय (199cm), गौरव (140cm), अशोक (179cm) - tallest?	संजय	राजेश (not in prompt!)
L	विकास (199cm), अनिता (140cm), नारायण (179cm) - tallest?	विकास	सीता (not in prompt!)

Table 8: What the mistakes look like.
Every wrong answer is still a real, well-formed name - the model just picks the wrong one, not a broken output.

When the two numbers being compared are close together, the models get it wrong more often. Correct answers had a much bigger gap between the two numbers on average (Hindi: 1289.9 vs. 525.4; Nepali: 1571.7 vs. 436.2) than wrong answers did - about 2.5× to 3.6× bigger.

3.4 Attention: Pretrained vs. Finetuned

	Hindi: pretrained --> finetuned	Nepali: pretrained --> finetuned
Mean entropy (lower = focused)	2.168 --> 1.932 bits (↓)	2.380 --> 2.020 bits (↓)
Mean distance (higher = long-range)	4.222 --> 4.690 (↑)	4.514 --> 5.269 (↑)

Table 9: Attention shift after reasoning finetuning.
Same comparative-reasoning prompt, before vs. after finetuning.

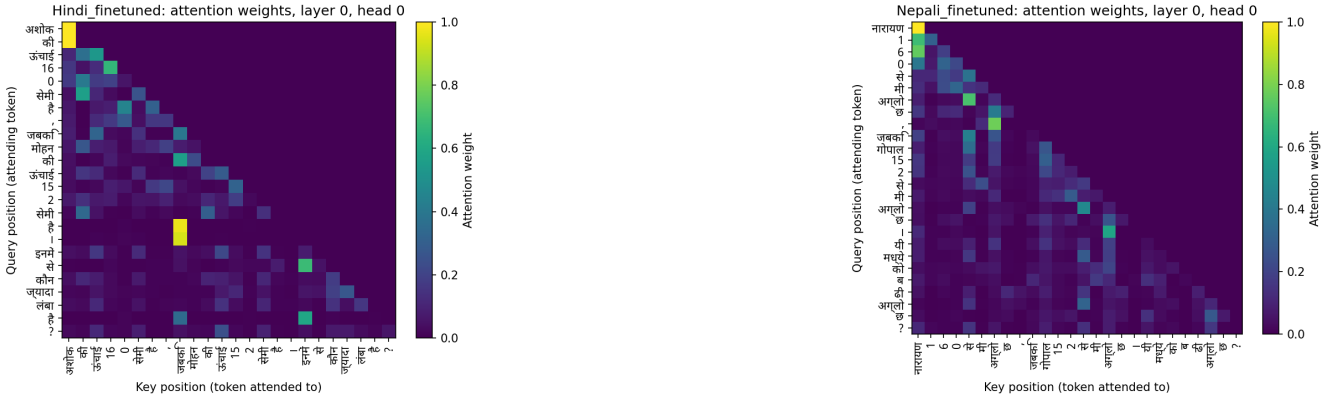


Figure 9: Layer 0, head 0, after finetuning: Hindi (left) vs. Nepali (right).

Observation: before finetuning, this same early head was almost purely diagonal - each word mostly looked at itself. After finetuning, both languages show new off-diagonal spikes to specific earlier tokens, as if the head learned to jump straight to a relevant number or name instead of only looking at itself.

Layer	Hindi distance shift	Nepali distance shift
0 (earliest)	+0.32	+0.20
1	−0.39	−0.12
2	+0.95	+0.96
3	+1.27	+0.76
4	+1.01	+0.19
5 (latest)	−0.34	**+2.54**

Table 10: Per-layer distance shift after finetuning, Hindi vs. Nepali.
Hindi's biggest change sits in the middle layers (2–3); Nepali's is almost entirely in the last layer (bold).

Finetuning makes attention *more focused* and *longer-range* in both languages: Hindi's biggest shift is in its *middle* layers, Nepali's almost entirely in its *last* (+2.54 at layer 5). Nepali's shift is, if anything, the bigger one despite lower accuracy - a weaker model limits its answers, not a failure to adapt.

4 Final Synthesis

4.1 Data Scale and Quality

The two final corpora ended up close in size (506.81M vs. 500.24M tokens), but only because Nepali needed extra manual effort: a whole additional Wikipedia crawl, and even then, more duplicate content turned up in its manual sources. The amount of distinct Nepali text freely available online is genuinely smaller than Hindi's, and matching the token count took real extra work.

4.2 Language-Modeling and Reasoning Results

On BPB and chrF++, the metrics that do not depend on the tokenizer, Hindi and Nepali are almost tied: under 0.5% and under 0.2 points apart. The real gap shows up in reasoning instead: Hindi's finetuned accuracy (65.0%) is well above Nepali's (43.4%).

4.3 Tokenizer and Corpus Effects on Nepali

Nepali needs more tokens per word than Hindi (1.4934 vs. 1.3227, same vocabulary size), which makes its raw perplexity look worse than it really is. Nepali's manually-collected sources also had noticeably more duplicate content, even after our deduplication steps.

4.4 What the Evidence Shows

The perplexity gap is a tokenizer side-effect, confirmed by BPB. The reasoning gap tracks with Hindi's small remaining edge in language quality. Both models adapted to finetuning the same way, and Nepali's attention shift was, if anything, the bigger one, just organized differently (Hindi in its middle layers, Nepali in its last). So the accuracy gap is not a failure to adapt: Nepali's weaker starting model just turns that same adaptation into correct answers less reliably.

Both languages also fail reasoning the same way: right format, wrong entity, worse on close values. This points to a shared limit of a small, 25M-parameter model at precise comparison, not something specific to either language.

5 Bonus (Optional): No-Positional-Embeddings Ablation

Section 2.1 explained *why* RoPE matters. This section tests that directly. We retrained Hindi from scratch with RoPE removed from `attention.py` - everything else about the model and training stayed exactly the same. RoPE has no learnable weights, so the parameter count does not change at all (still 25,019,328). This means only one thing changed between the two models.

Metric	Standard	Ablated	Change
Perplexity / BPB	24.24 / 0.4665	27.23 / 0.4836	mild (+12% / +3.7%)
BLEU-4 / chrF++ / ROUGE-L	38.34 / 48.94 / 0.242	4.21 / 25.38 / 0.0	**collapse**
Distinct-1 / Distinct-2	0.300 / 0.548	0.571 / 0.792	higher, but not a real gain

Table 11: Standard model vs. no-positional-embeddings ablation.
Same data, same training budget, only the positional signal removed.

What broke, and what did not. Guessing one likely next word from nearby words does not really need word order, so perplexity and BPB barely move. But BLEU, chrF, and ROUGE-L all check whether the generated text matches a reference *in the right order* - and all three collapse. Watching the model write text shows why: it repeats *"लखनऊ लखनऊ लखनऊ लखनऊ"*, the same word four times in a row, because without any sense of position, it cannot tell what it just said a moment ago. This also explains an odd result: Distinct-1/2 actually go *up*. Those scores only count unique word combinations across the whole sample - they do not notice this kind of local repeating, so a model can look "more varied overall" and still be stuck repeating itself, at the same time.

The model does not just ignore the missing signal - it visibly changes to cope with the loss. Layer 0 becomes even more tightly focused on nearby words (distance drops from 0.885 to 0.440, nearly half). Layers 2 and 3 do the opposite, spreading out further (0.895 to 1.131, and 1.102 to 1.223), as if searching more broadly to make up for what RoPE would normally give it for free. That adjustment is real - but nowhere near enough to fix generation quality.

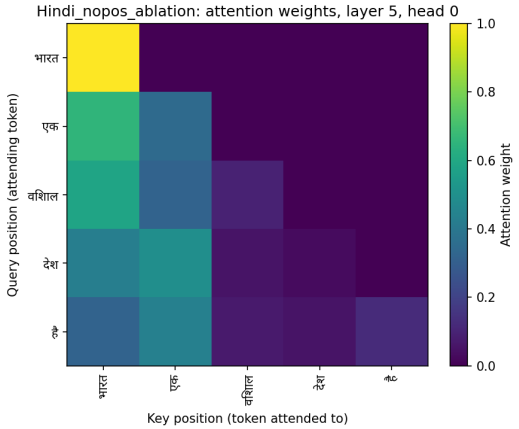


Figure 10: Layer 5, head 0, with no positional embeddings.
Observation: this shape is still strikingly close to the standard model's own late-layer heatmap (Figure 7), even with zero explicit positional signal. The reason is that causal masking alone leaks a weak, hidden clue about position: a query simply has more keys available to look at the later it sits in the sentence.

In short: a model does not need position to guess one word, but it needs position to hold a whole sentence together. RoPE is not decoration - it is doing real work.